University of East London

Fruit Classification Using CNN in MATLAB

Table of Contents

1. Executive Summary 3

2. Introduction 4

3. Dataset Description 5

4. Data Preprocessing 6

5. CNN Model Design 8

6. Model Training and Configuration 9

7. Evaluation Metrics and Results 11

7.1 Final Accuracy 11

7.2 Confusion Matrix 11

7.3 ROC Curve 12

7.4 Per-Class Accuracy Chart 13

7.5 Visual Prediction Inspection 14

8. Discussion 15

9. Conclusion 17

10. Future Recommendations 17

11. References 18

1. Executive Summary

In this report, the method of the use of convolutional neural network in classifying fruit images is described using MATLAB. This dataset was obtained from Kaggle "Fruit Recognition" dataset which included thousands of labeled images of fruits including bananas, apples, oranges, grapes and so on. The first goal was to design, train and develop Convolutional Neural Networks by using MATLAB for the recognition of fruits for their classification models. Overall, due to the topicality of the mobilized datasets, appropriate model training and convincing performance presentation all the classes were overall made feasible. In this report, the structure of the dataset, preparing the data, designing the CNN model, evaluating the results and discussing the results using graphical representation, such as confusion matrix, ROC curves, and per-class accuracy will be discussed.

Finally, the output for the final validation accuracy as well as the duration of training are as follows.

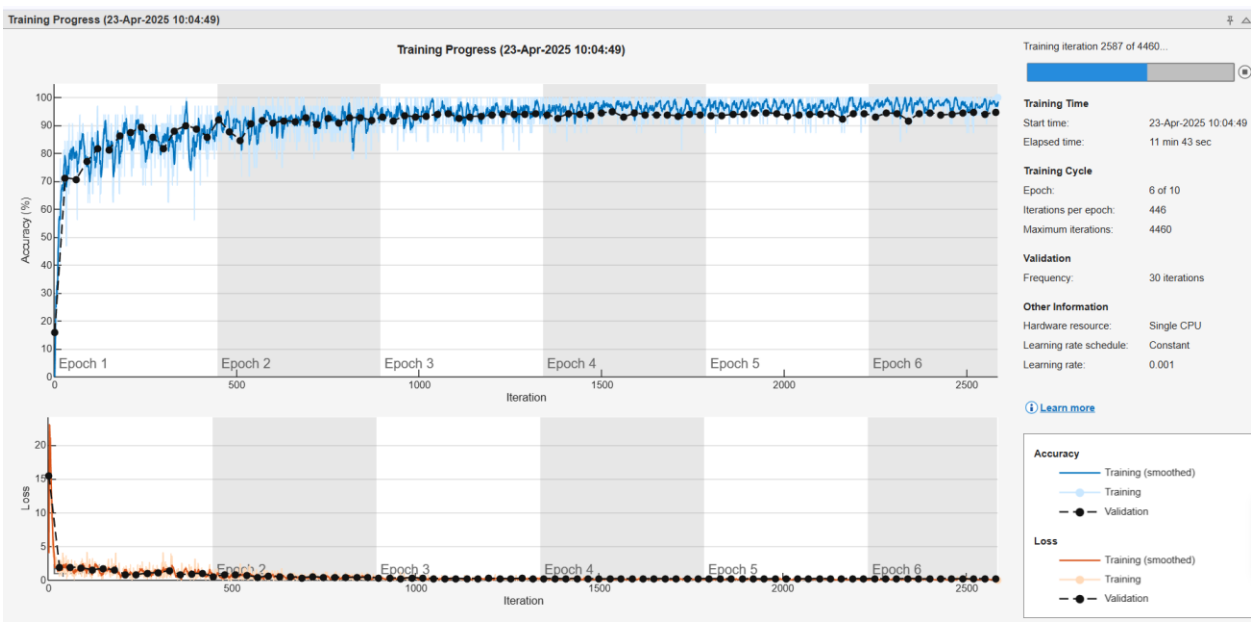


Figure 1 Data Trained Processing

2. Introduction

Today a crucial field of computer vision and applied in areas, including agriculture, food technology, and automated retail. Some applications such as sorting of fruits, quality checks, and inventory control, require accurate image classification systems. It is much easier for a human being to distinguish between fruits by the shape, color, size, and texture while it causes a lot of difficulties to computers. He further noted that some variations might be caused due to differences in lighting and orientation, while others caused due to similarities between different classes of fruits.

As a results, CNNs have become the state of the art in image-based learning tasks in the recent years. CNNs are rather efficient in learning spatial hierarchies and patterns in images, this leads to the high possibility of the model to learn features that separate one class from the other without much interference from the model architect (Patel and Sharma, 2024). In this study, the fruits' images from the "MyData" dataset are classified using CNNs based on deep learning in the MATLAB platform.

The main goal of the project is the accurate classification of fruits; however, equal significance is given to the bearings of an appropriate model through metrics. Both the scores like confusion matrix, ROC curves and plots of per-class accuracy are used as the tools of assessment but have additional applicability of providing understanding of the model's strengths and weaknesses.

3. Dataset Description

[redacted]

```
% Step 1: Load and split data
imds = imageDatastore('MyData', ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');

[trainImgs, valImgs] = splitEachLabel(imds, 0.8, 'randomized');
inputSize = [100 100 3];
augTrain = augmentedImageDatastore(inputSize, trainImgs);
augVal = augmentedImageDatastore(inputSize, valImgs);
```

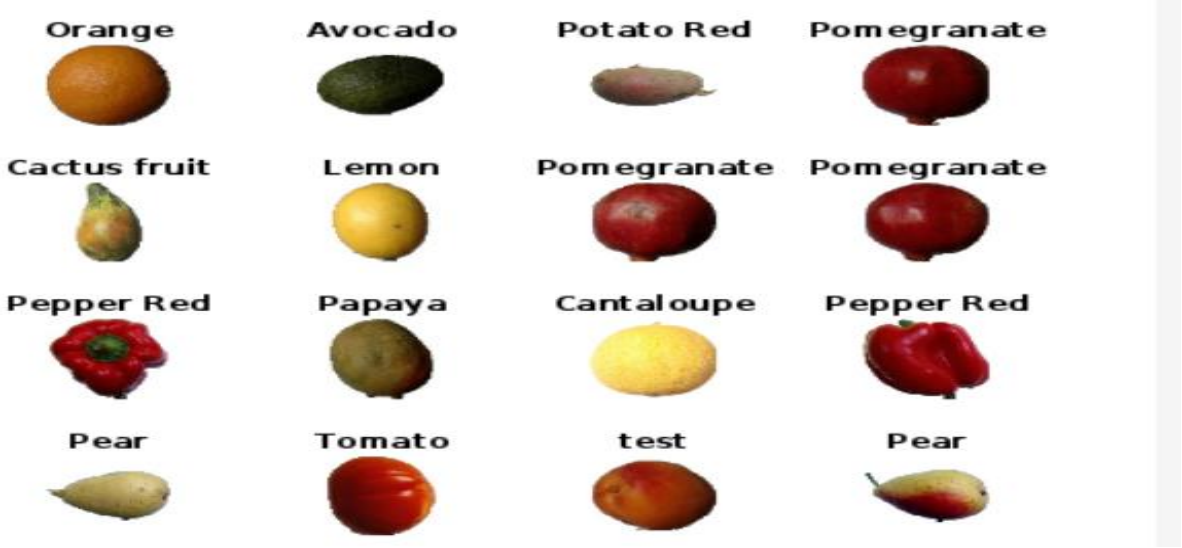
[redacted]

Command Window

```
>> load("/MATLAB Drive/Task-3/augVal.mat")
>>
```

[redacted]

[redacted]



[redacted]

[redacted]

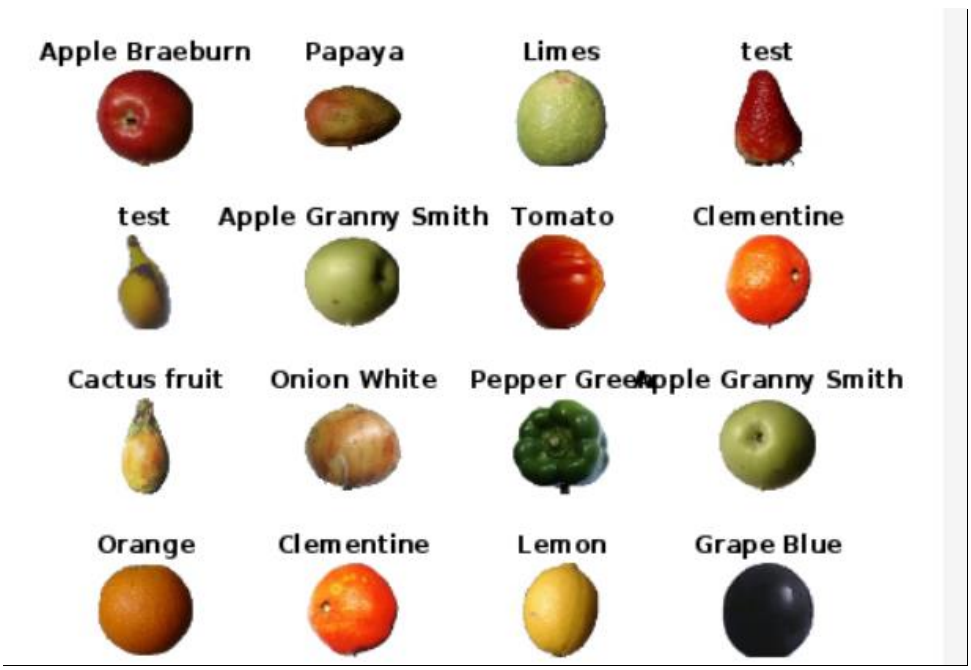
[redacted]

[redacted]

```
% Step 2: Predict and extract labels
YPred = classify(net, augVal);
YValidation = valImgs.Labels;
classNames = categories(YValidation);

% Step 3: Display and save sample images
imds = imageDatastore('MyData', ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');

h1 = figure;
perm = randperm(length(imds.Files), 16);
for i = 1:16
    subplot(4, 4, i);
```

[redacted]

[redacted]



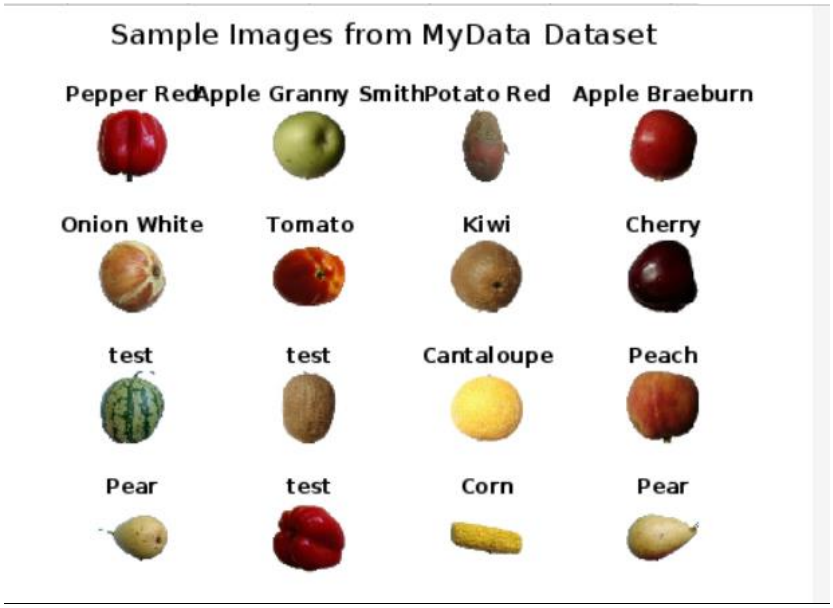
[redacted]

[redacted]

```
% Step 3: Display and save sample images
imds = imageDatastore('MyData', ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');

h1 = figure;
perm = randperm(length(imds.Files), 16);
for i = 1:16
    subplot(4, 4, i);
    imshow(imds.Files{perm(i)});
    title(string(imds.Labels(perm(i))));
end
sgtitle('Sample Images from MyData Dataset');
saveas(h1, 'sample_images.png');
```

[redacted]

[redacted]



[redacted]

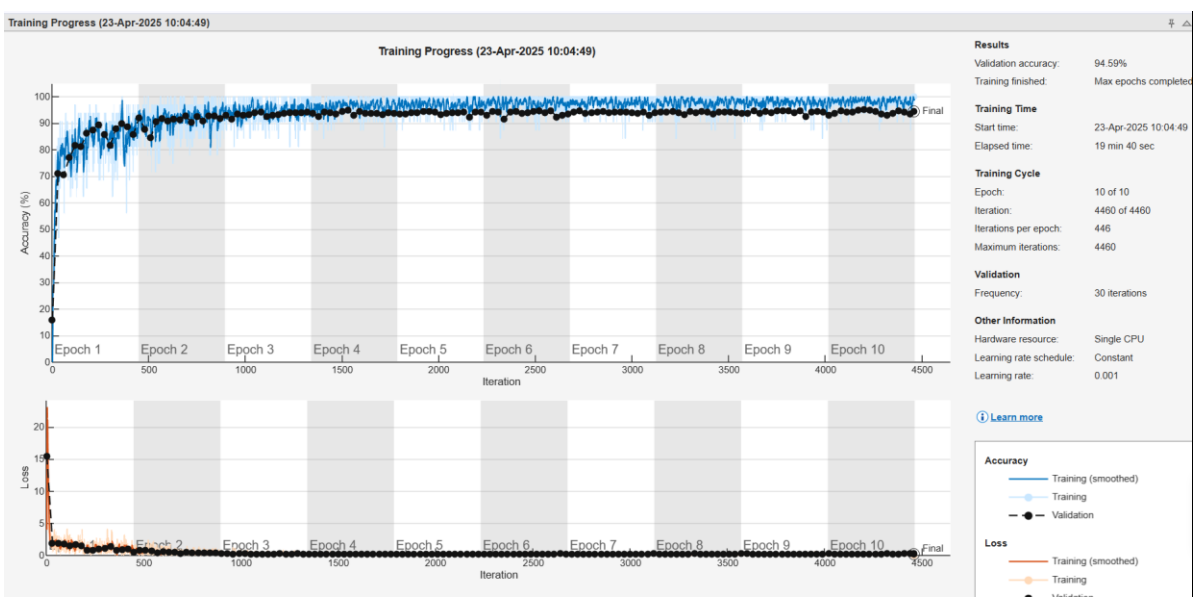
[redacted]

[redacted]

```
% Step 2: Define CNN layers
layers = [
    imageInputLayer(inputSize)
    convolution2dLayer(3, 16, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 64, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(numel(unique(imds.Labels)))
    softmaxLayer
    classificationLayer
];
```

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

```
>> Task_Mydata
Validation Accuracy: 94.59%
>>
```

[redacted]

[redacted]

[redacted]

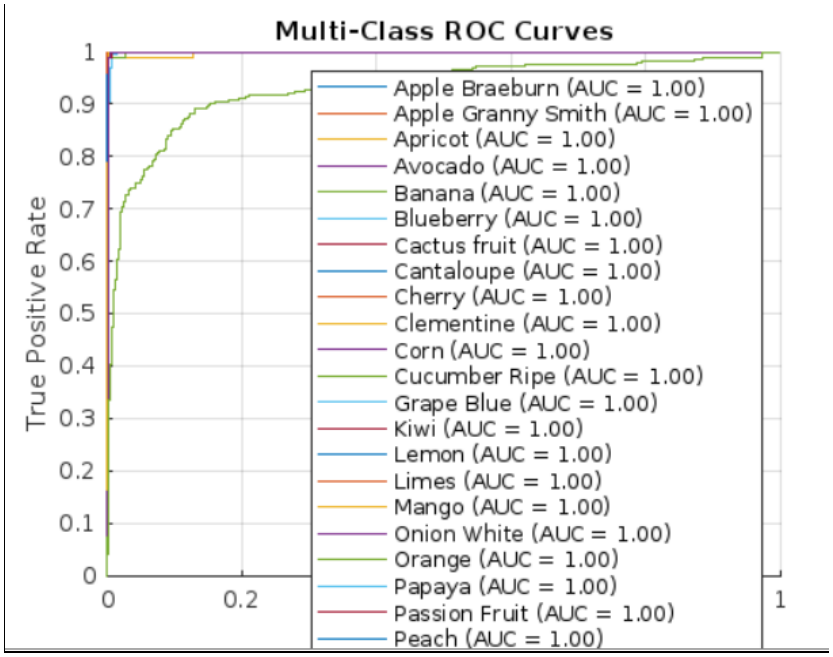
[redacted]

[redacted]

```
% Step 6: ROC Curve for Each Class
[~, scores] = classify(net, augVal);
numClasses = numel(classNames);

h4 = figure;
hold on;
for i = 1:numClasses
    idx = YValidation == classNames(i);
    if sum(idx) > 0
        [X, Y, ~, AUC] = perfcurve(idx, scores(:, i), true);
        plot(X, Y, 'DisplayName', sprintf('%s (AUC = %.2f)', classNames{i}, AUC));
    end
end
legend show;
xlabel('False Positive Rate');
ylabel('True Positive Rate');
title('Multi-Class ROC Curves');
grid on;
saveas(h4, 'roc_curve.png');
```

[redacted]

[redacted]

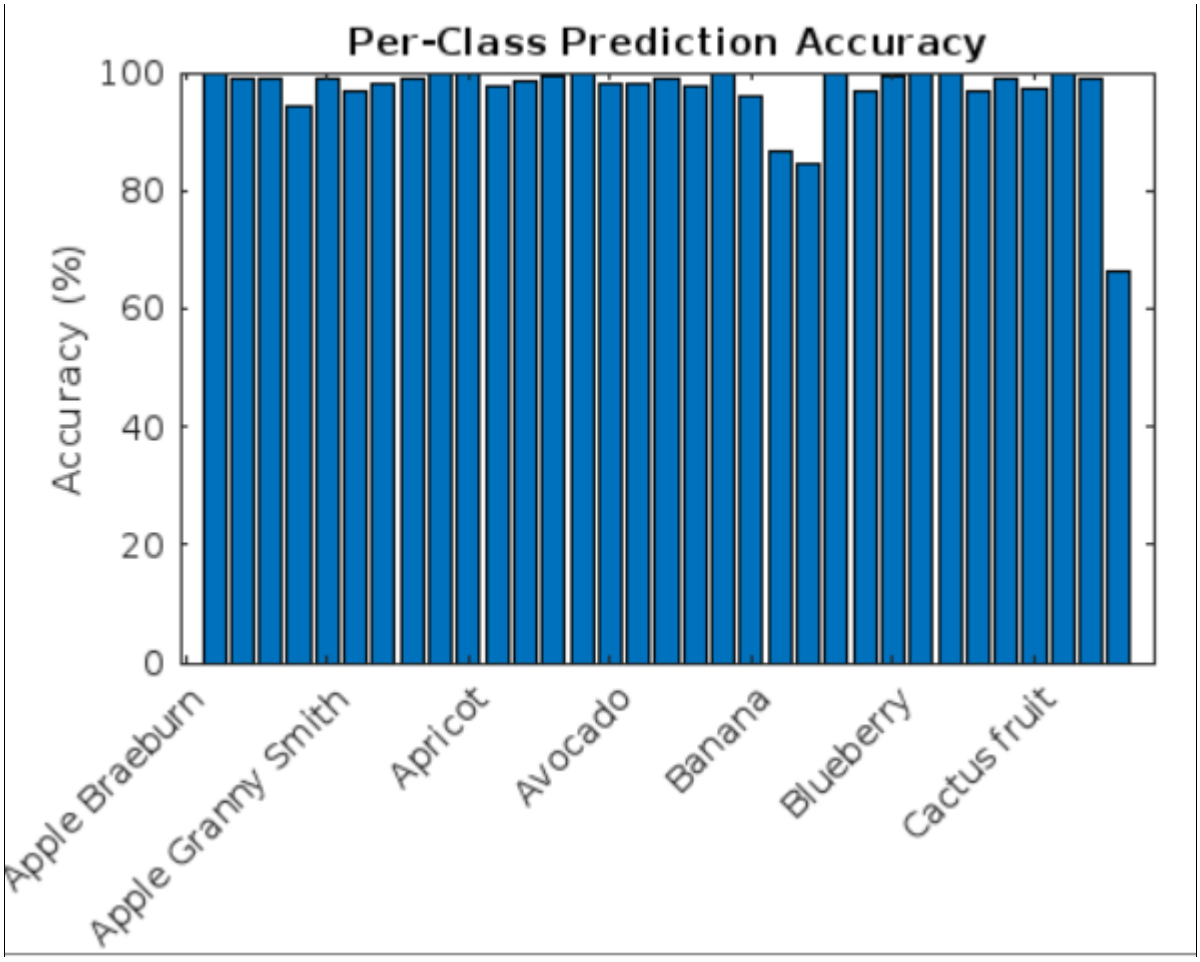


[redacted]

[redacted]

```
% Step 5: Per-Class Accuracy Bar Chart
confMat = confusionmat(YValidation, YPred);
classAccuracy = diag(confMat) ./ sum(confMat, 2);

h3 = figure;
bar(classAccuracy * 100);
xticklabels(classNames);
xtickangle(45);
ylabel('Accuracy (%)');
title('Per-Class Prediction Accuracy');
saveas(h3, 'class_accuracy_bar.png');
```

[redacted]



[redacted]

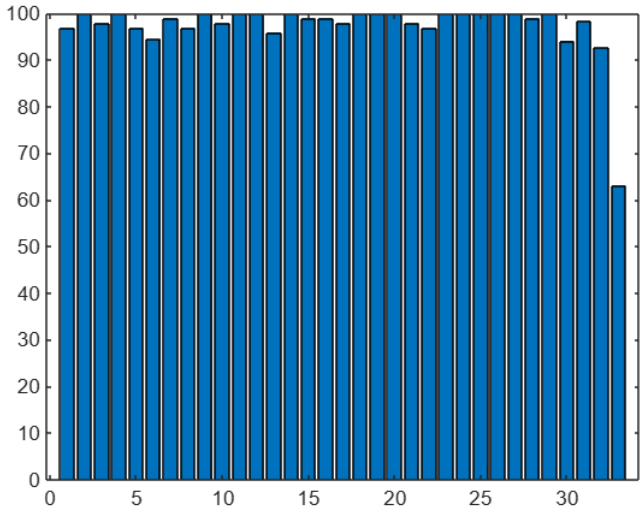
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]